

02 | LLM Architecture - Week 04 | Tokenizers, RNN and the Attention Mechanism – Summary

25.04.2026

The lecture covers foundational concepts in Natural Language Processing (NLP) crucial for understanding Large Language Models (LLMs), focusing on how text is processed into a usable format, how sequential data is handled, and an advanced mechanism for improved performance.

Tokenization

Tokenization is the process of breaking down raw text into "tokens," which replace the traditional concept of "words" for LLM processing. The initial pipeline involves raw text undergoing a pre-processing phase, often called standardization, which can include removing punctuation or lemmatization (reducing words to their base form).

Comparison of Tokenization Levels: The lecture discusses two basic tokenization methods and their trade-offs:

- **Character-Level Tokenization:** Each character is its own token and has its own embedding.
 - **Pros:** No out-of-vocabulary (OOV) problem (any word can be composed from characters), naturally handles misspellings and new morphologies.
 - **Cons:** Creates very long sequences, which is more costly and slower for LLMs (as costs are often per token). Lacks built-in semantic meaning. Vocabulary size is very small.
- **Word-Level Tokenization:** Breaks sentences into words, where each word is a token.

- **Pros:** Shorter sequence length, more efficient training. Tokens correspond to semantic units, making meaning easier to learn. Vocabulary size is very large.
- **Cons:** Significant OOV issues for unknown words or misspellings unless the vocabulary is huge or specific fallback mechanisms exist.

Subword Tokenization (Better Solutions): To overcome the limitations of character and word-level tokenization, subword tokenization aims to find a middle-ground solution by breaking words into meaningful subword units (e.g., "raining" into "rain" and "ing").

Three main algorithms are presented:

1. Byte Pair Encoding (BPE):

- **Mechanism:** Starts with all characters in the training data as base tokens. It then iteratively counts the frequency of adjacent token pairs and merges the most frequent pair into a new token. This process repeats until a predefined vocabulary size (a hyperparameter) is reached.
- **Example:** Given a corpus with words like "hug," "pug," "pun," "bun," and "hugs," and a target vocabulary size of 10. Initially, individual characters (h, u, g, p, n, b, s) form the vocabulary. The pair "UG" is identified as the most frequent (20 times) and merged into a new token "ug". The process continues, for example, merging "U" and "N" into "UN" (16 times) over "H" and "UG" (15 times).
- **Inference:** When tokenizing new words, the algorithm always looks for the longest possible token first. If a character or subword is not in the vocabulary, it's mapped to an [UNKNOWN] token (e.g., "mug" becomes [UNKNOWN] and "ug" if "m" is not in vocab, "thug" becomes [UNKNOWN] and "hug" if "t" is not).
- **Application:** Used in models like GPT and RoBERTa.

2. WordPiece:

- **Mechanism:** Similar to BPE in that it learns merge rules, but the selection criteria differ. Instead of the most frequent pair, WordPiece computes a **likelihood score** for each pair: $(\text{frequency_of_pair}) / (\text{frequency_of_first_element} \times$

frequency_of_second_element). This prioritizes pairs where the elements are individually less frequent but form a highly frequent combination.

- **Distinction:** WordPiece differentiates between a letter at the beginning of a word and one in the middle (e.g., using ## prefix for internal characters).
- **Example:** While "UG" might be the most frequent pair, its elements "U" and "G" might also be very frequent individually, leading to a lower score than another less frequent pair whose elements are rare on their own. For instance, a pair like "GS" might have a higher score because "G" and "S" are less frequent individually compared to "U".
- **Application:** Used by BERT and Electra.

3. SentencePiece:

- **Mechanism:** Operates in reverse compared to BPE/WordPiece. It starts with a very large initial vocabulary (potentially generated by BPE or WordPiece) and gradually **removes** tokens. At each step, it estimates how much the negative log likelihood (loss function, similar to logistic regression's cross-entropy) of the training data would increase if a particular token were removed. Tokens that cause the smallest increase in loss (meaning their removal has the least negative impact on model performance) are pruned.
- **Example:** If removing "PU" results in the same loss as keeping it (because "pug" could still be tokenized as "P" and "UG" with the same score), it's a candidate for removal. Conversely, removing "HUG" might significantly worsen the loss, making it a token to keep.
- **Input Handling:** A key differentiator is that SentencePiece works directly with raw text, treating spaces as characters. This means it doesn't rely on pre-tokenization by spaces, which is beneficial for languages without explicit word delimiters (like Chinese or Japanese) or for handling multilingual inputs where character similarity between languages can be an issue.
- **Application:** Used by T5, XLNet, and ALBERT.

General Tokenization Considerations:

- **Special Tokens:** Common practice to add special tokens like [UNKNOWN] (for out-of-vocabulary items), [EOS] (end-of-sentence), and [CLS] (for classification tasks, especially in models like BERT).
- **Vocabulary Size:** Typically ranges from 32,000 to 64,000 tokens, depending on factors like multilingual support. This is a **hyperparameter** chosen based on performance metrics like sequence length reduction, perplexity, or final task performance.
- **Domain Specificity:** Custom tokenization for specific domains (e.g., healthcare) can significantly improve meaning capture and reduce sequence length and computational cost (training and inference). For example, in the medical domain, "hypertension" might be one token in a PubMed-trained BERT but split into "hyper" and "tension" in a general BERT.
- **Morphologies and Typos:** Subword tokenization helps handle misspellings and morphological variations more naturally than word-level tokenization by preserving meaningful parts of a word.
- **Numbers:** LLMs generally struggle with numbers, treating them as accumulations of characters and often memorizing specific numerical patterns rather than understanding their mathematical semantics. Complex calculations are typically offloaded to external tools like Python.
- **Implementation:** Hugging Face offers a `tokenizers` library and an interactive tokenizer playground to experiment with different LLM tokenizers.
- **Training Implications:** Tokenization is a computationally intensive, one-time process performed *before* LLM training. If a new tokenization is desired for an existing model, retraining the model from scratch is generally necessary due to its foundational role. Algorithms like BPE, WordPiece, and SentencePiece are greedy, meaning extending an existing vocabulary suboptimally might not yield the best results.

Recurrent Neural Networks (RNNs)

RNNs are architectures particularly well-suited for processing **sequential data**, offering an organic way to handle inputs of varying lengths, unlike traditional neural networks.

Characteristics of Sequential Data:

- **Vocabulary:** Composed of distinct items or "tokens" (e.g., words in text, phonetics in audio, characters in DNA).
- **Time Dependence:** Elements often have a temporal order (e.g., audio, video, stock prices).
- **Locality:** The position of an element within the sequence is important.
- **Long-Distance Relations:** Information at the beginning of a sequence can be relevant to elements much later in the sequence (e.g., sentence structure, DNA).

RNN Mechanism: An RNN takes a sequence of inputs ($X_1, X_2, \dots, X_N$) and an initial state (S_0). At each step i :

- **State Calculation:** The current state s_i is a function (e.g., $\tanh$) of the previous state s_{i-1} and the current input x_i .
- **Output Calculation:** The current output y_i is a function of the current state s_i .
- **Parameter Sharing:** Crucially, the same set of parameters (θ s) are shared across all steps of the sequence, making the model compact and enabling it to learn patterns that apply regardless of position in the sequence.
- **Initial State (S_0):** Can be a zeroed vector or can be used to provide context (e.g., encoding style, or the representation of an input sentence in an encoder-decoder architecture).

RNN Architectures:

- **Acceptor:** Uses only the **final state** (S_N) or output (Y_N) of the RNN, which encapsulates information from the entire input sequence. This fixed-size representation is then used for tasks like **text classification** (e.g., sentiment analysis).

- **Transducer:** Generates an **output for each token** in the input sequence. This is useful for tasks requiring per-token labeling, such as **Named Entity Recognition (NER)** (identifying entities like persons, organizations) and **Part-of-Speech (POS) tagging** (identifying verbs, nouns, etc.).
 - **Generative Transducer (Language Model):** A special case where the RNN predicts the next token in a sequence. It takes the output of the previous step as an input for the current step, allowing it to generate non-Markovian sequences (predictions depend on the entire history, not just the immediately preceding token). Generation stops when an [EOS] token is predicted. Can be forward (predicting next word) or backward (predicting previous word).
- **Encoder-Decoder:** Combines an **acceptor (encoder)** to process an input sequence into a fixed-size context vector, and a **transducer (decoder)** to generate an output sequence from that context vector. This is highly effective for tasks like **language translation** (e.g., English to French), **text summarization**, and **tone change**. The encoder's final state (or some other representation) serves as the *s₀* for the decoder.

Advanced RNNs:

- **Deep RNNs:** Involve multiple layers, allowing for more complex representations.
- **Bidirectional RNNs:** Process the input sequence in both forward and backward directions, then concatenate the resulting hidden states. This allows the model to leverage information from both past and future contexts for each token.
- **LSTM (Long Short-Term Memory):** A specific type of RNN that was state-of-the-art before transformers, designed to address the vanishing gradient problem in vanilla RNNs.

Training RNNs:

- **Labels:** The labels depend on the task. For sentiment analysis, it's a single label (positive/negative). For NER/POS, it's a label for each token. For language models, the label is the *next* word in the sequence.

- **Hyperparameters:** The size of the hidden state (h) and the output dimension (y) are hyperparameters that need to be tuned for optimal performance.
- **Parallelization:** Training can be parallelized using batching (processing multiple sequences at once), but sequences within a batch need to be padded to the maximum length. The sequential nature of RNNs means that predictions depend on previous outputs, limiting full parallelization across time steps, but the parameters are updated all at once via backpropagation for a batch.

Attention Mechanism

The attention mechanism emerged to address a significant limitation of RNNs, particularly the difficulty of encoding the full meaning of very long sentences into a single fixed-size vector. This "bottleneck" meant RNNs struggled to selectively focus on the most relevant parts of a sequence when making predictions.

Core Idea: Instead of compressing all information into one vector, the attention mechanism uses **multiple vectors** to represent the sequence. It allows the model to dynamically decide which parts of the input sequence are most relevant for predicting a current output. This is achieved by having a "query" vector score "key" vectors.

How Attention Works:

1. **Query (Q):** Represents the current hidden state or the element for which attention is being calculated (e.g., the word being predicted in a language model).
2. **Keys (K):** A set of vectors representing all previous hidden states or inputs in the sequence. In a forward RNN, keys are states preceding the query.
3. **Values (V):** Often the same as the key vectors (previous hidden states or input embeddings), but can sometimes be different.
4. **Attention Scores:** The query vector is multiplied (typically a dot product) with each key vector to determine their similarity or relevance. These scores indicate how much attention the current query should pay to each past element.
5. **Softmax:** A softmax function is applied to these attention scores. This normalizes them into a probability distribution (weights) ranging from 0 to 1, summing to 1.

These weights explicitly indicate the importance of each past element to the current query.

6. **Context Vector (Attention Output):** The attention weights are used to compute a **weighted average** of the value vectors. This results in a single, fixed-size "attention output" vector that contains aggregated information from the entire sequence, with more emphasis on the elements identified as important by the query. This output is then combined with the current input and previous state to form the new hidden state.

Benefits of Attention:

- **Adaptive Focus:** Allows the model to adapt its focus to specific, relevant parts of the input sequence based on the current task or prediction step, rather than relying on a general representation.
- **Improved Performance:** Significantly boosts the performance of sequence models for tasks like machine translation and language generation.
- **Interpretability:** Attention weights can be visualized (e.g., in an attention plot for machine translation, showing which source words influenced which target words), providing some insight into the model's decision-making process, though its role as a direct "explanation" is debated.
- **Building Block for Transformers:** The attention mechanism is a fundamental component of the Transformer architecture, which will be discussed in the next lecture.

The lecture concludes by noting that while RNNs are good with sequences, they are no longer the state-of-the-art, and the attention mechanism is a crucial step towards understanding the Transformer architecture, which is now the dominant method for sequences.